

Problem A. Shopping Fever

Purchasing two items together is never needed: you can instead purchase each of them separately and get the same discount.

Purchasing more than three items together is also never needed: you can instead purchase just three of them together and the rest separately and you will pay *at most* the same as for the original purchase.

Thus, there is always an optimal solution in which we purchase some items in groups of three and the rest individually.

In Subtask 1 it is therefore sufficient to check only two cases: either we purchase all three items together, or we purchase each item separately.

In Subtask 2 we have $q = 0$ and thus the only way to get a discount is to get some items for free. In order to get an item for free, we need to buy it together with (at least) two items that are at least equally expensive. From this observation it can easily be shown that the optimal solution in this case is to sort the item costs in descending order and purchase the items in that order, in groups of three.

In Subtask 3 we have $q = 40\%$. Note that with promotion #1 we can save at most one third of the total price of the purchased items (which is about 33.3%). Hence, whenever q is at least 34, the optimal solution is to purchase all items one at a time, always getting the q percent discount.

We are left with cases where q is between 1 and 33, inclusive. In these cases we might be tempted to solve the task greedily, too, but simple greedy strategies aren't always optimal in these cases.

For example, suppose the discount is $q = 10$ percent and the item costs are 1000, 900, 100, 100, 100. If you purchase $1000 + 900 + 100$ together as in the solution to Subtask 2, you will get the item worth 100 for free, but for these three items you only saved five percent of their total price. This is because the third item was much cheaper than the other two. It would have been better to buy each of them separately. But that's still not optimal. For this test case, the optimal solution is to buy the items worth 1000 and 900 separately, and to purchase the other three items together to save 100 on them.

Let's think some more about the structure of optimal solutions. In Subtask 2 it was optimal to always buy three consecutive items together. Can we generalize this somehow?

Sure we can. Suppose the item costs are $p_1 \geq p_2 \geq \dots p_{n-1} \geq p_n$. We now claim that we can always construct an optimal solution such that whenever we buy three items together, they are at consecutive positions in this sorted sequence.

First, suppose we already decided which subset of items will be purchased in groups of three and which ones will be purchased separately. For those purchased in groups of three we have the same situation as in Subtask 2: we know that the optimal solution is to form the groups of three in the order of costs. Thus, it is enough to look for a solution in which the groups of three don't overlap in the sorted order of costs.

Second, suppose we are purchasing items i, j, k together and $k \neq i+2$. Instead of doing so, we can purchase $k-2, k-1, k$ together and each element in the range $[i, k-3]$ separately. We will still get element k for free, and in the new solution the individual items that get the discount are at least as expensive as in the old solution.

At this moment we therefore know that there is always an optimal solution in which each purchase of three items is a purchase of three consecutive items in the sorted order. In other words, in this optimal solution we take the sorted sequence of items and divide it into blocks of sizes 1 and 3.

The observation we just made allows us to find the best solution of this form using *dynamic programming*. We will process the prices in decreasing order. In each step i we will compute the minimum cost C_i we have to pay in order to buy the items with prices $p_1, \dots, p_i$. We start with $C_0 = 0$. In step i we consider the problem of purchasing the first i items optimally. There are two options: either item i is purchased alone, or (for $i \geq 3$) the triple $i-2, i-1, i$ is purchased together.

The cheapest solution of the first type has cost $C_{i-1} + (100 - q) * p_i / 100$ because we get item i at a discount and we need to purchase the remaining $i-1$ items as cheaply as possible. The cheapest solution of the second type has cost $C_{i-3} + p_{i-2} + p_{i-1}$. The optimal solution is obtained by picking the cheaper

option, so C_i equals the minimum of those two costs.

The final answer is the value C_n . The total time complexity of this solution is $O(n \log n)$ due to sorting, the memory complexity is $O(n)$.

Problem B. Railway

Subtask 1

Loop all triples (train j from Zurich, train k from Lugano, Tunnel i) and check if the two trains crash in the tunnel. We can do this check in $\mathcal{O}(1)$ time as shown below, so this takes $\mathcal{O}(nmt)$ time in total.

A train from Zurich departing at time c_j enters the tunnel i at time $c_j + a_i$, and leaves it at time $c_j + b_i$. A train from Lugano to Zurich departing at time d_k enters tunnel i at time $d_k + (s - b_i)$ and leaves it at time $d_k + (s - a_i)$.

Since the two trains travel in opposite directions, they crash in the tunnel if they are both strictly inside the tunnel at some point in time. Thus, it suffices to check if the **real** intervals $(c_j + a_i, c_j + b_i)$ and $(d_k + (s - b_i), d_k + (s - a_i))$ intersect.

Subtask 2

Loop over tunnels, and for every tunnel, compute the union A of the intervals of time when trains from Zurich are strictly inside the tunnel, and the union B of times trains from Lugano are strictly inside it. If $A \cap B \neq \emptyset$, there is a crash in the tunnel. We can check if the intersection is empty in $\mathcal{O}(n + m)$ time, for $\mathcal{O}((n + m)t)$ total work.

Subtasks 3 and 4

Remember the condition for trains j and k to crash inside tunnel i : $(c_j + a_i, c_j + b_i)$ and $(d_k + (s - b_i), d_k + (s - a_i))$ must intersect. This holds if $c_j + a_i < d_k + (s - a_i)$ and $d_k + (s - b_i) < c_j + b_i$. Moving terms, we get $c_j - d_k < s - 2a_i$ and $s - 2b_i < c_j - d_k$. Thus, we must have $c_j - d_k \in (s - 2b_i, s - 2a_i)$.

This gives a $\mathcal{O}(nm \log t)$ solution: let T be a sorted vector of the tunnel intervals $(s - 2b_i, s - 2a_i)$. Loop all pairs (train j from Zurich, train k from Lugano), and binary search the first interval for which $s - 2b_i < c_j - d_k$, then check if $c_j - d_k < s - 2a_i$. If true, the two trains crash in the tunnel. Otherwise, the trains do not crash.

The difference between subtasks 3 and 4 is that, as long as s, c_j and d_k are all even, all trains meet at integer coordinates. This does not change the solution, but can make implementing some approaches easier. You can also guarantee that s, c_j and d_k are even by multiplying all input values by 2.

Problem C. Angry Cows

Subtask 1

We have $n \leq 10$, and so we can afford to try every possible set of walls. For each such set, we can check the (non-)connectivity requirements using a BFS or DFS algorithm: starting from some hiking areas, and making sure not to walk on walls, we check whether we have visited all hiking areas but no cow-areas. We also need to find the remoteness of each area (which can be done using Dijkstra's algorithm) and the overall remoteness of every possible wall set, picking the one with minimum remoteness.

Subtask 2

In this subtask we will make what is possibly the most important observation for the entire task. Namely, the intuition is that, rather than surrounding the hiking areas, it is better to surround the cow areas, as this will give us the highest chance of keeping the hiking areas connected to each other. In general, this may not result in the smallest possible remoteness, but in this subtask we do not care, as everything has remoteness 0.

So: we just place walls on all unused areas that neighbor a cow-area! This wall set, which we call W , will surely separate hikers from cows (unless there is a cow-area adjacent to a hiking area, in which case we should always print -1). However, we still need to verify that we have not disconnected some hiking areas. As in subtask 1, we can do this by running a BFS or DFS traversal from some hiking area, avoiding W , and then check whether all hiking areas have been visited. If everything is fine, we can print W . On the other hand, if some hiking areas are disconnected, then W is infeasible. However, in this case in fact no set of walls is feasible, and we can print -1! Why is that?

Assume there exists some feasible set W' of walls. Then for any two hiking areas there is some path connecting them that avoids W' . Crucially, this path cannot use any area in W either, because if it did, this would create a W' -avoiding path from either of these hiking areas to a cow-area (that the W -area used by the path is adjacent to). This shows that if there is some feasible set W' of walls, then there are W -avoiding paths between any two hiking areas, which makes W feasible as well.

Subtask 3

One *incorrect* solution that suggests itself here is to place walls in all neighboring areas of the single hiking area h . However, there are two issues with this approach:

- Some of these walls might be unnecessary (since no cow-area can be reached from them without passing through h). Including them might unnecessarily increase the overall remoteness.
- Even taking the above into account, this set of walls might still not have minimal remoteness. This is because there can exist two unused areas such that the first is a neighbor of h whereas the second is not (so the first is closer to h if we look at unweighted distances), but the second is closer in terms of remoteness (weighted distances from h). If walling either of these areas would separate h from cow-areas, then we should select the second rather than the first.

In order to solve this subtask, we need to make the second crucial observation in the task. Namely, we can perform a binary search over the overall remoteness of the solution, which will reduce the task to checking, for a given r , whether there exists a wall set with remoteness at most r .

To do this, in this subtask it is enough to check whether placing walls in all unused areas of remoteness at most r disconnects the hiking area from all cow-areas. Clearly, if this happens for some r , then it also happens for all larger r ; in this subtask we are not worried about disconnecting hiking areas from each other. To find the remotenesses of all areas, we run Dijkstra's algorithm.

Subtask 4

Let us root the tree at some hiking area h . We can use dynamic programming (a DFS traversal of the tree) to compute, for each v , a binary value $h_v \in \{0, 1\}$ that is equal to 1 if the subtree rooted at v contains some hiking area. This can be done in linear time. (For hiking areas v we set $h_v = 1$; for other areas, if they are leaves, then $h_v = 0$, and otherwise h_v is the binary disjunction (OR) of all h_w for w

being children of v .)

If there is a cow-area v with $h_v = 1$, then there is no good wall set, as this cow-area lies on the (unique) path between h and some hiking area. Also, as usual, if a cow-area is adjacent to a hiking area, then there is no solution. Otherwise, the set S of areas v with $h_v = 1$ is a connected set, containing the root h , that must be kept connected (i.e. we cannot place any walls there). We should place walls along the outside boundary of S : these are unused areas and we will thus disconnect S (and all hiking areas) from cow-areas.

However, we should not place walls on *all* boundary areas of S , as some of them might be unnecessary (they do not actually separate S from any cow-area). To know which ones to select, we can e.g. compute values $c_v \in \{0, 1\}$ for every area v : $c_v = 1$ if the subtree rooted at v contains some *cow*-area. This is done in exactly the same way as for h_v . Then, we would only place walls on boundary areas of S that have $c_v = 1$.

Subtask 5

Here we need a fully general solution as for Subtask 6 (see below), but it may check all remotenesses iteratively (instead of running a binary search), or its implementation may be inefficient (e.g. Dijkstra or BFS/DFS not running in nearly-linear time).

Subtask 6

We use the binary search idea introduced for Subtask 3. This reduces the problem to determining, for a given r , whether there is a wall set of remoteness at most r . A wall set will have remoteness at most r if and only if it consists of areas with remoteness at most r . Therefore, once we have computed all remotenesses using Dijkstra's algorithm, we can divide all areas into: cow-areas, hiking areas, allowed unused areas, and forbidden unused areas. The task is to find any wall set that only uses allowed unused areas.

Now that we no longer care about remoteness, the remaining task looks very similar to Subtask 2 – with the difference that now there are forbidden areas where we cannot place walls. This prevents us from just walling in the cows (as in the solution to Subtask 2).

However, notice that a forbidden area that is adjacent to a cow-area can be effectively treated like a cow-area (as we must disconnect it from all hiking areas). The same applies to a forbidden area adjacent to a forbidden area adjacent to a cow-area, and so on. We can expand in this way until we find the outside-boundary areas where we should place the walls. More precisely, if we consider the graph obtained by looking only at cow-areas and forbidden areas, then we should place walls at the outside boundary of every connected component of this graph *that contains a cow-area*.

If any of these outside-boundary areas happens to be a hiking area, then there exists a path of the form “cow-area – zero or more forbidden areas – hiking area”, so the current choice of r is infeasible. Otherwise, the cows are successfully surrounded, and we need to check whether the hiking areas are still connected (using BFS/DFS – see Subtask 1 or 2). If yes, we return success into the binary search (the current value of r is feasible). If not, we return failure – and we can do this for the same reason as in Subtask 2.

Problem D. Double Move

There are two parts of this problem:

- For the given set of complete choices (i.e., for $k = n$), find the number of assignments in which Alice wins.
- Run a brute force search on all decisions made by both players to determine which move is optimal, and prune this search to speed it up.

Let's tackle the first part. Consider an initially empty graph on n vertices, numbered 1 through n . Denote this graph by G_0 . Recall the i -th declaration (numbered from 1) is a pair a_i, b_i . To build graph G_i , we add an undirected edge $\{a_i, b_i\}$ to G_{i-1} . Note that these graphs may contain multiple edges or loops.

We model the choice made by Claire by orienting each edge towards the vertex corresponding to Claire's choice. That is, an edge $\{a_i, b_i\}$ is substituted by an arc (a_i, b_i) if b_i is selected by Claire and (b_i, a_i) otherwise. Again, this graph may contain multiple edges, loops or edges in opposite directions. In order for the game to continue, the in-degree of each vertex must be at most 1.

Let G be an undirected graph. We define $f(G)$ to be the number of such orientations of graph G . Clearly, $f(G_0) = 1$ and $f(G_1) = 2$. The following values, however, depend on the structure of G_i . Let's learn how to evaluate f .

Consider connected components of f . Clearly, the decisions made in one component don't influence the other. Hence, we can split G into connected components, evaluate f in each of them and then multiply the results.

A connected graph G on m vertices contains at least $m - 1$ edges. Let's break this down based on the actual number of edges.

- If G contains exactly $m - 1$ edges (i.e, it is a tree), there are exactly m valid orientations. To see why this is true, observe that we can root the tree in the only vertex that has in-degree 0, and each edge is oriented away from the root.
- If G contains exactly m edges, then it is a tree plus a single edge. We can orient this additional edge in an arbitrary way. Note that this uniquely determines all remaining edges (as we can reduce this to the previous case where one vertex must have in-degree 0). This means there are exactly 2 ways of orienting the edges in such a graph.
- If G contains at least $m + 1$ edges, there are more edges and vertices. By pigeonhole principle, it is impossible to orient the edges such that the indegree of each vertex is at most 1.

Let $g(i)$ be the number of Claire's choices that end the game after exactly i turns. Clearly, the number of games won by Alice is $g(2) + g(4) + g(6) + \dots$.

We claim that for i from 1 to $n + 1$, we have $g(i) = f(G_{i-1}) \times 2^{n+2-i} - f(G_i) \times 2^{n+1-i}$. Let's prove this.

Consider G_i . The first i of Claire's decisions are modelled by orienting the edges of G_i , while the remaining $n + 1 - i$ decisions are not. This means that for every fixed orientation of G_i , there are exactly 2^{n+1-i} possible Claire's choices that lead to this directed graph. A similar argument can be made for G_{i-1} . Now, note that the assignments that end the game after exactly i moves are those that have a corresponding valid orientation in G_{i-1} , but not in G_i . This concludes the proof.

Now that we know how to compute the answer for a given set of choices of Alice and Bob, let's concentrate how to find the answer when $k < n$. We will do this by employing the so called minimax algorithm. This algorithm works as follows.

Every list of between 0 and $n + 1$ player declarations is called a **game state** s . The value $h(s)$ is called the **evaluation** of the game state, and it represents the number of Claire's choices that result in Alice's win, given that both sides play optimally **from this state onwards**. Clearly, if s contains exactly $n + 1$ player

declarations, then the game is finished and $h(s) = g(2) + g(4) + g(6) + \dots$. To compute the evaluation for a state s that isn't final, consider the set $S(s)$ of all states reachable in one move, i.e., states that append exactly one declaration to s . Then, for Alice's turn

$$h(s) = \max_{s' \in S(s)} h(s')$$

and for Bob's turn

$$h(s) = \min_{s' \in S} h(s')$$

Put together the answer for a starting state s (if Alice starts) is

$$h(s) = \max_{s^{(1)} \in S(s)} \min_{s^{(2)} \in S(s^{(1)})} \max_{s^{(3)} \in S(s^{(2)})} \dots h(s^{(n-k)})$$

The name minimax is coined by this alternation of minimum and maximum.

With all these bits, we can finally discuss the solution.

Subtask 1

In the first subtask, the limits are very small, and we can simply try all possible move sequences. Since each of them can be evaluated in $\mathcal{O}(n)$, and there are $\binom{n}{2}^{n+1}$ of them, the solution runs in $\mathcal{O}(n^{2n+3})$ time. Note that this solution doesn't need the observation on how to compute $f(G)$ by inspecting the connected components of G , and we can use brute force for that step, yielding a solution with running time $\mathcal{O}(n^{2n+2} \cdot 2^n)$.

In the following subtasks, our goal is to reduce the number of states considered. Note that we do not expect the contestants to be able to compute how exactly does the number of states change with each optimisation, but rather to intuitively understand why such optimisations reduce the number of states. Nevertheless, the concepts used in the analysis contain useful topics for contestants' further study.

Subtask 2

Note that $f(G)$ doesn't depend on the actual graph structure, but only on its connected components and whether it contains $m - 1$, m or more edges. That is, we only need to store the connected components and their type. Additionally, any component of the third type (more than m edges) makes answer equal to 0, regardless of further declarations, and we can break immediately.

As there are only two component types, there are

$$\sum_{i=0}^n 2^i \left\{ \begin{matrix} n \\ i \end{matrix} \right\}$$

states, where $\left\{ \begin{matrix} n \\ i \end{matrix} \right\}$ is the Stirling number of the second kind, or the number of ways to partition n elements into exactly i non-empty sets. For $n = 10$, this number is 4 412 798, which yields a fast enough algorithm.

Subtask 3

For subtask 3, notice that after processing the initial declarations, the vertices within a connected component are interchangeable. In other words, we only need to remember the number of vertices and the number of edges in each component. Furthermore, when enumerating choices for adding an edge, we may simply add an edge within a component, or merge two components together.

How many game states do we have? We no longer partition a set of distinguishable elements, but a set of indistinguishable elements. As such, the number of partitions is dictated by the Partition number $p(n)$ – the number of ways of expressing n as a sum of positive integers, regardless of their order. This quantity still grows exponentially with n , but much slower than the previous one.

To pinpoint the number of states, we split the vertices into those that are in the component of first type and those that are in the component of second type, and then partition each separately. Thus, the number of states is

$$\sum_{i=0}^n p(i) \cdot p(n-i)$$

where $p(i)$ is the Partition number. For $n = 25$, this quantity is 127 554.

Subtask 4

For subtask 4, notice that there aren't that many possible test inputs as $k = 0$. We can thus run the solution for all cases locally, and submit just the hardcoded answers. The reference solution for the subtask 3 finishes in about two minutes for $n = 35$ and $k = 0$ on an average machine.

Subtask 5

The final observation needed is that we don't need to know the sizes of the components of the second type. This is because the answer doesn't depend on their size, and when we add an edge to them in any way (either by adding an edge within the component, or by connecting it to another component), no component of the first type can be created. Thus, it is sufficient to remember the sizes of components of the first type, and the number of the components of the second type. The total number of states is now

$$\sum_{i=0}^n p(i) \cdot (n-i+1)$$

For $n = 35$, this quantity is 403 779.